MeTMaP: Metamorphic Testing for Detecting False Vector Matching Problems in LLM Augmented Generation

Guanyu Wang*
Beijing University of Posts and Telecommunications

Yuekang Li*
University of New South Wales

Yi Liu
Nanyang Technological University

Gelei Deng
Nanyang Technological University

Tianlin Li
Nanyang Technological University

Guosheng Xu†
Beijing University of Posts and Telecommunications

Yang Liu
Nanyang Technological University

Haoyu Wang
Huazhong University of Science and Technology

Kailong Wang†
Huazhong University of Science and Technology

ABSTRACT

Augmented generation techniques such as Retrieval-Augmented Generation (RAG) and Cache-Augmented Generation (CAG) have revolutionized the field by enhancing large language model (LLM) outputs with external knowledge and cached information. However, the integration of vector databases, which serve as a backbone for these augmentations, introduces critical challenges, particularly in ensuring accurate vector matching. False vector matching in these databases can significantly compromise the integrity and reliability of LLM outputs, leading to misinformation or erroneous responses. Despite the crucial impact of these issues, there is a notable research gap in methods to effectively detect and address false vector matches in LLM-augmented generation.

This paper presents MeTMaP, a metamorphic testing framework developed to identify false vector matching in LLM-augmented generation systems. We derive eight metamorphic relations (MRs) from six NLP datasets, which form our method's core, based on the idea that semantically similar texts should match and dissimilar ones should not. MeTMaP uses these MRs to create sentence triplets for testing, simulating real-world matching scenarios. Our evaluation of MeTMaP over 203 vector matching configurations, involving 29 embedding models and 7 distance metrics, uncovers significant inaccuracies. The results, showing a maximum accuracy of only 41.51% on our tests compared to the original datasets, emphasize the widespread issue of false matches in vector matching methods and the critical need for effective detection and mitigation in LLM-augmented applications.

CCS CONCEPTS

• **Security and privacy → Software security engineering**.

*Equal Contribution

†Corresponding authors: guoshengxu@bupt.edu.cn and wangkl@hust.edu.cn

FORGE '24, April 14, 2024, Lisbon, Portugal

ACM ISBN 979-8-4007-0609-7/24/04…$15.00
https://doi.org/10.1145/3650105.3652297

KEYWORDS

Metamorphic Testing, Vector Matching, Augmented Generation

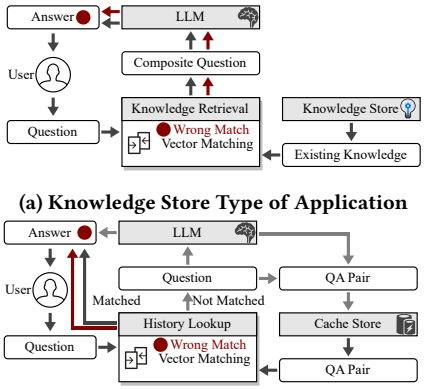
ACM Reference Format:
Guanyu Wang, Yuekang Li, Yi Liu, Gelei Deng, Tianlin Li, Guosheng Xu, Yang Liu, Haoyu Wang, and Kailong Wang. 2024. MeTMaP: Metamorphic Testing for Detecting False Vector Matching Problems in LLM Augmented Generation. In *AI Foundation Models and Software Engineering (FORGE '24), April 14, 2024, Lisbon, Portugal.* ACM, Lisbon, Portugal, 12 pages. https://doi.org/10.1145/3650105.3652297

1 INTRODUCTION

The landscape of large language models (LLMs) has been profoundly reshaped by the advent of augmented generation techniques, particularly Retrieval-Augmented Generation (RAG) and Cache-Augmented Generation (CAG). These advancements have leveraged the capabilities of vector databases, specialized for storing and retrieving vector representations from embedding models, to significantly enhance LLM functionalities. In Figure 1, systems such as the Cache Store (e.g., GPTCache [86]) and the Knowledge Store (e.g., Langchain [6]) exemplify this integration, showcasing how these databases bolster LLMs. By efficiently caching and indexing crucial information, they not only streamline the retrieval process but also empower LLMs to deliver more accurate, current, and contextually relevant responses.

The incorporation of vector databases into LLM frameworks holds immense potential for improving information retrieval efficiency. However, it also introduces challenges, particularly in accurate vector matching. The implications of these challenges manifest as two primary issues: miss matching and false matching. Miss matching leads to a fallback on standard information retrieval methods, thus forgoing the efficiency of integrated vector databases. More critically, false matching—akin to a web cache erroneously delivering an unrelated webpage—can significantly impair LLM functionality, either by yielding irrelevant query results or by selecting inappropriate background information, potentially leading to misleading or fabricated model outputs.

Despite the critical need to accurately identify false matching for optimal LLM application functionality, current testing methodologies for vector matching methods remain underdeveloped. This gap is further compounded by the absence of a systematic benchmark for evaluating these methods, a challenge exacerbated by the lack of reliable test oracles.



(a) Knowledge Store Type of Application

(b) Cache Store Type of Application

Figure 1: Workflow of two LLM-based QA applications.

Addressing this research gap, our paper introduces MeTMaP, a metamorphic testing framework specifically designed to detect false matching problems in LLM-augmented generation. We derive eight metamorphic relations from six NLP task datasets, categorized into Word-Level and Sentence-Level relations based on metamorphosis degree. MeTMaP leverages these relations to structure test cases as triplets: a base sentence, a semantically similar positive sentence with a different structure, and a negative sentence with an inverse meaning. These triplets, generated using AI models, are then used to simulate and test the vector matching methods in LLM-augmented generation applications.

Our comprehensive evaluation of MeTMaP spans 203 vector matching configurations, combining 29 embedding models and 7 distance metrics. The results are revealing: a significant decrease in accuracy on our generated test cases compared to original datasets, with the highest accuracy being only 41.51%. We further conduct a case study on three real-world open source vector databases [11, 24, 62], and show that MeTMaP can detect false matching problems in LLM-augmented generation systems. The highest accuracy for correct vector matching is only 37.72%. This underscores a widespread and non-negligible issue of false matches across all evaluated vector matching methods and highlights the structural bias over semantic accuracy in these systems.

Contribution. The key contributions are enumerated as follows:

- **Innovative Metamorphic Testing Framework for LLM Augmented Generation.** We introduce a groundbreaking framework for evaluating vector matching in LLM augmented generation, marking a first in the field to the best of our knowledge. A key contribution is the development of eight semantic-based metamorphic relations at the sentence level. These relations are crucial for benchmarking LLM augmentation techniques.
- **Comprehensive Evaluation of Matching Methods.** Our study conducts a thorough evaluation of 203 vector matching methods, combining various embedding models and distance metrics. This extensive analysis offers crucial insights into the performance and limitations of current matching techniques in LLM contexts.
- **Insights into False Matching in Augmented LLMs.** A significant finding is the tendency of vector matching methods to prioritize structural over semantic similarities, contrasting with the more effective semantic discernment of text matching methods. This highlights areas for improvement in matching techniques in LLM augmented generation.

We have open-sourced the dataset and more detailed experiment results at [1] to facilitate open-science and future research.

2 BACKGROUND

2.1 Vector Databases

A vector database is a specialized database designed to handle vector data, which are essentially multi-dimensional arrays or lists of numerical values often used to represent features in machine learning models, images, or any other data with multiple attributes. Unlike traditional databases that handle scalar values like integers or strings, vector databases are optimized for high-performance similarity search, allowing users to quickly retrieve the most similar vectors to a given query vector based on various distance or similarity metrics like Cosine Similarity or Euclidean Distance.

Key features of a vector database often include scalability, low-latency retrieval, and support for multiple similarity metrics. They are also designed to handle massive datasets and can be distributed across multiple nodes to improve search performance. Advanced vector databases may also offer features like auto-indexing, query-time filtering, and integration with machine learning frameworks, thus making them particularly useful in applications ranging from recommendation systems to computer vision and natural language processing. The major concern, however, is the accuracy of the information retrieval, which is the key focus of this work.

2.2 Metamorphic Testing

Metamorphic testing [9] serves as a dual-purpose technique for both generating test cases and verifying test results, strategically addressing the so-called "oracle problem" in many software testing scenarios [8, 45, 58, 71, 77, 83]. At the core of this methodology lies the concept of metamorphic relations (MRs), key properties that delineate the expected associations between sets of input-output pairs in the software being tested. The procedure entails taking an initial test case and applying a predefined transformation rule to produce a closely related new test case. Subsequently, the outputs of these paired test cases are scrutinized to confirm their adherence to the established MRs. Since its inception, the domain of metamorphic testing has witnessed substantial scholarly activity, spanning a range of focus areas including the identification of MRs, innovative test case generation strategies, integration with existing software engineering paradigms, and rigorous validation and evaluation of software systems. In particular, we propose MRs for textual contents to test the semantic similarity for queries in LLM-integrated applications (Figure 1) in this work.

3 METHODOLOGY

In this section, we introduce our MeTMaP. The overview of our testing framework is shown in Figure 2. We aim to construct a corpus where each sample is a triplet of the following form:

$$\{Query : Base \mid Candidates : Positive, Negative\} \quad (1)$$

Semantically, the base sentence and the positive are considered identical sentences, but contradictory to the negative. In terms of sentence structure, the structural similarity between base and negative sentences is higher than with positive sentences. We first

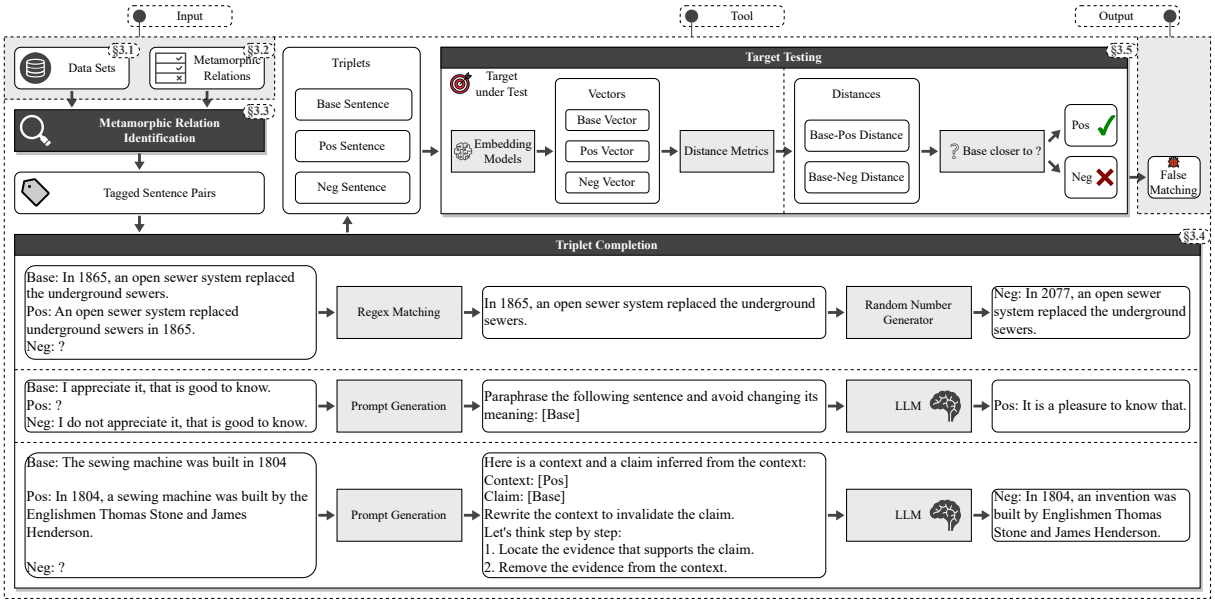


Figure 2: MeTMaP overview with three parts: sentence pairs collection, triplet completion, and vector matching simulation of.

conduct a pilot study to look for metamorphosis that may provoke changes at the semantic level (subsection 3.1). Then, we introduce eight MRs from the pilot study in subsection 3.2 and collect sentence pairs from six existing datasets in subsection 3.3. We complement the sentence pairs into triplets in subsection 3.4 and show how to use them for testing in subsection 3.5.

3.1 Pilot Study

In this work, we intend to explore how to make semantic changes with as few changes to a sentence as possible. To design sufficiently effective metamorphosis, we first conduct a pilot study on existing datasets to explore which metamorphosis meet our expectation. We consider the following 6 sentence relation datasets, including **Stanford Contradiction Corpora [14]**, **PAWS [84]**, **VitaminC [59]**, **Inference-is-Everything [75]**, **HEROS [10]** and **NEVIR [74]**.

To answer our question, we collect samples labelled as contradictory etc. indicating non-consistent relationships between sentence pairs. Then, we analyse the structure of these samples. Specifically, We collect 500 sentences pairs from each of the datasets. We iterate through each pairs, looking for differences in the sentences. We only consider samples that contain more identical words. This is because in such cases, sentence pairs are more likely generated by some kind of morphing rather than by a random combination of two unrelated sentences. We categorise the metamorphic methods into word-level and sentence-level based on the extent of the metamorphosis. Word-level includes Word Swap-based, Object Substitution-based, Action Substitution-based, Negative Expression-based, Word Deletion, Quantifier Substitution-based. Their metamorphosis occurs only on a particular word or phrase. Sentence-level includes Back Translation-based and Inference Generation-based. They usually produce more changes in sentence structure. We summarise eight MRs accordingly (to be detailed in Section 3.2).

3.2 Metamorphic Relation

In this section, we introduce eight MRs inspired by the pilot study. To facilitate the unambiguous understanding, we formally define them. We further classify them into two types based on the degree of metamorphosis: **Word-Level** metamorphosis and **Sentence-Level** metamorphosis. In particular, we first define the base sentence, from which we derive the negative sentence using metamorphic transformation (subsection 3.3). Then, the positive sentences are completed by the generator (subsection 3.4).

3.2.1 ***Word-Level Metamorphosis.*** In this level, we choose sentence pairs that result in semantic contradictions by deleting, replacing, or swapping the order of the words. The samples at this level are characterized with very similar sentence structures, and only individual words are metamorphosed. Specifically, given a base sentence, we aim to derive the corresponding negative sentence using the MR.

Definition 3.1. **(Base Sentence)** The base sentence can be defined by an N-tuple:

$$\text{Base sentence} = < w_1, w_2, ..., w_t, ..., w_{n-1}, w_n >,$$

where $w_k (0 \leq k \leq n)$ represents a constituent word in the sentence, w_t represent a target word.

Word Swap. This MR yields a contradiction by reordering the words.

Definition 3.2. **(Word Swap)** Given the base sentence:

$$\text{Base sentence} = < w_1, w_2, ..., w_t, ..., w_{t+j}, ..., w_{n-1}, w_n >,$$

where w_t and w_{t+j} are two target words. Then we can obtain the negative sentence:

$$\text{Negative sentence} = < w_1, w_2, ..., w_{t+j}, ..., w_t, ...w_{n-1}, w_n >,$$

Object Substitution. This MR replaces a word or phrase in a sentence that can refer to an object to create an inconsistency, which are usually nouns or pronouns.

Definition 3.3. **(Object Substitution)** Given the target word w_t being an object, the negative sentence is given:

$$\text{Negative sentence} = < w_1, w_2, ..., w_{obj}, ..., w_{n-1}, w_n >,$$

Table 1: Metamorphic Relations

Metamorphic Levels	Relation Names/Shortcuts	Examples	Sources
Word Level	Word Swap (Word Swap, WS)	***Base:*** The only industry in the town is light farming on the small rice paddies. ***Posi:*** Light farming on the small rice paddies is the only industry in the town. ***Nega:*** The light industry in the town is only farming on the small rice paddies.	***Base:*** Collect ***Posi:*** Generate ***Nega:*** Collect
	Object Substitution (Obj Sub, OS)	***Base:*** Carl borrowed a book from Richard, but the book was unreadable to him. ***Posi:*** Carl was unable to make sense of the book he borrowed from Richard. ***Nega:*** Carl borrowed a book from Richard , but the book was unreadable to Richard.	***Base:*** Collect ***Posi:*** Generate ***Nega:*** Collect
	Action Substitution (Act Sub, AS)	***Base:*** I think we know what we 're going to speak about. ***Posi:*** I believe we are aware of what to discuss. ***Nega:*** I think we be what we 're going to speak about.	***Base:*** Collect ***Posi:*** Generate ***Nega:*** Collect
	Negative Expression (Nega Exp, NE)	***Base:*** I appreciate it, that is good to know. ***Posi:*** It is a pleasure to know that. ***Nega:*** I do not appreciate it, that is good to know. ***Base:*** But it is Jackie saying it so sorta disappointing. ***Posi:*** Jackie saying it makes it somewhat disappointing. ***Nega:*** But it is Jackie saying it so sorta upbeat.	***Base:*** Collect ***Posi:*** Generate ***Nega:*** Collect
	Word Deletion (Word Del, WD)	***Base:*** In 2012, Jordan started all 16 games while recording 8.0 sacks and 54 tackles. ***Posi:*** In 2012, jordan started 16 games and recorded 8.0 sacks and 54 tackles. ***Nega:*** ~~In 2012~~, Jordan started all 16 games while recording 8.0 sacks and 54 tackles.	***Base:*** Collect ***Posi:*** Generate ***Nega:*** Collect
	Quantifier Substitution (Quant Sub, QS)	***Base:*** In 1865, an open sewer system replaced the underground sewers. ***Posi:*** An open sewer system replaced underground sewers in 1865. ***Nega:*** In 3016 , an open sewer system replaced the underground sewers.	***Base:*** Collect ***Posi:*** Generate ***Nega:*** Generate
Sentence Level	Back Translation (Err Trans, ET)	***Base:*** The second series was well received by the critics better than the first. ***Posi:*** The critics received the second series more favorably than the first. ***Nega:*** The first series was recorded by critics better than the second.	***Base:*** Collect ***Posi:*** Generate ***Nega:*** Collect
	Inference Generation (Err Nli, EN)	***Base:*** The sewing machine was built in 1804. ***Posi:*** In 1804 , a sewing machine was built by the Englishmen Thomas Stone and James Henderson , and a machine for embroidering was constructed by John Duncan in Scotland. ***Nega:*** In 1804, an invention was built by Englishmen Thomas Stone and James Henderson, and a device for embroidering was constructed by John Duncan in Scotland.	***Base:*** Collect ***Posi:*** Collect ***Nega:*** Generate

where w_{obj} represents a different object compared with the original target word.

Action Substitution. This MR replaces an action described in a sentence to create a contradiction.

Definition 3.4. **(Action Substitution)** Given the target word w_t being a verb, the negative sentence is given:

$$\text{Negative sentence} = < w_1, w_2, ..., w_{verb}, ..., w_{n-1}, w_n >,$$

where w_{verb} represents a different action represented by the original target word.

Negative Expression. This MR adds negatives to the front of the verb or uses antonyms to replace adjectives in base sentence to produce semantic contradictions.

Definition 3.5. **(Negative Substitution)** Given the target word w_t being a verb, adverb or adjective, the negative sentence is given:

$$\text{Negative sentence} = < w_1, w_2, ..., w_{neg}, ..., w_{n-1}, w_n >,$$

where w_{neg} represents one or a set of semantically negative word(s) versus the original target word.

Word Deletion. This MR deletes words or phrases in base sentence to generate contradictions. One point worth noting is that this category of samples is not completely dissimilar to the other categories. The reason is that deleting a part of a sentence may amplify the range of meanings expressed in the sentence. This means that it can infer metamorphic sentences from original sentences, but not feasible in the opposite direction (as listed in Table 1, we can infer the negative sentence from the base sentence, but in turn we can't infer which year the events described in the negative sentence happen in). We denote this phenomenon as *One-Way Inconsistency*, which will bring a new challenge to semantic evaluation.

Definition 3.6. **(Word Deletion)** Given the target word w_t, the negative sentence is given:

$$\text{Negative sentence} = < w_1, w_2, ..., remove(w_t), ..., w_{n-1}, w_n >,$$

where $remove(w_t)$ represents the deletion of the original target word.

Quantifier Substitution. This MR generates contradiction by identifying and changing the quantifier in a sentence.

Definition 3.7. **(Quantifier Substitution)** Given the target word w_t being a quantifier, the negative sentence is given:

$$\text{Negative sentence} = < w_1, w_2, ..., w_{quantifier}, ..., w_{n-1}, w_n >,$$

where $w_{quantifier}$ represents a different quantifier compared with the original target word.

3.2.2 ***Sentence-Level Metamorphosis.*** In this level, the metamorphosis of the base sentence is no longer limited to a particular word or phrase. That means base and negative sentences have not only different meanings but even more different structures in this level. Due to the diverse patterns and possibilities for generating the triplet, we utilize informal definition to describe the MRs.

Back Translation. This MR records changes in sentence meaning due to back translation errors. It can be seen as a free combination of **Word-Level** MRs.

Inference generation. This MR produces contradictory sentence pairs through incorrect NLI. Similarly, *One-Way Inconsistency* exists in this category. That is, it can infer information about the base sentence from the positive sentence, but the inference doesn't stand up in the opposite direction.

3.3 MR Identification

Algorithm 1: Word-Level Metamorphosis Tagging

```
   Data: Sentence Pair < S1, S2 > and label from the existing datasets
   Result: Tag
1  Function GoldTag(< S1, S2 >, label):
2      if label is "Entailment" then
3          Tag ← "Other"
4      else
5          W1 ← S1.Lower.Split;
6          W2 ← S2.Lower.Split;
7          if W1.Sort = W2.Sort then
8              Tag ← "WordSwap"
9          else
10             U ← W1 ∪ W2;
11             if |W1| = |W2| and (|U| − |W1| ≤ 1 or |U| − |W2| ≤ 1) then
12                 diff1, diff2 ← FindDiff(W1,W2);
13                 if diff1 = Regex(S1) and diff2 = Regex(S2) then
                       // Regex match quantifiers
14                     Tag ← "QuantSub"
15                 else
16                     tag_dict1, tag_dict2 ← NLTK(S1, S2);
17                     tag1 ← tag_dict1[diff1];
18                     tag2 ← tag_dict2[diff2];
19                     if tag1 ⊂ [NN, PRP, JJ, RB, VB] and
                         tag2 ⊂ [NN, PRP, JJ, RB, VB] then
                           // Identify the part of speech. NN and
                              similar terms represent word parts of
                              speech.
20                         if tag1 ⊂ [NN, PRP] and tag2 ⊂ [NN, PRP]
                             then Tag ← "ObjSub" ;
21                         else if tag1 ⊂ [JJ, RB] and tag2 ⊂ [JJ, RB] then
                             Tag ← "NegaExp" ;
22                         else if tag1 is "VB" and tag2 is "VB" then
                             Tag ← "ActSub" ;
23                         else Tag ← "Other" ;
24                     else
25                         Tag ← "Other"
26             else
27                 if W1 ⊂ W2 then
                       // Assuming |W1| < |W2|
28                     if |W2| − |W1| ≤ 2 and "not" in W2 − W1 then
29                         Tag ← "NegaExp"
30                     else
31                         Tag ← "WordDel"
32                 else
33                     Tag ← "Other"
34     return Tag
35 Function FindDiff(W1, W2):
36     for w1, w2 in W1, W2 do
37         if w1 ≠ w2 then
38             return w1, w2
39 Function Regex(Sentence):
40     quant ← findall("−?\d + (? : \.\d+)?");
41     return quant
42 Function NLTK(S1, S2, diff1, diff2):
43     tag_dict1 ← nltk.word_tokenize(S1);
44     tag_dict2 ← nltk.word_tokenize(S2);
45     return tag_dict1, tag_dict2
```

In this section, we show given a sentence pair, how to determine whether it matches our MRs and with which MR it matches.

We use the algorithm 1 to distinguish Word-Level MRs: Given a sentence pair and its relation label, we first determine whether they are in an inconsistent relation (line 2,3). Then we convert both sentences to lower case to eliminate case effects and split them into two sets of words, W1 and W2. (line 5,6). When the sorting of W1 and W2 is consistent, it indicates the inconsistency in the pairs resulted from swapping of words (line 7,8). If two sentences are the same length and there are inconsistencies due to word substitution, we first traverse W1 and W2 to determine which word has been substituted (line 11,12). Once the word is found, we use a regular expression to determine if the metamorphosis word is a quantifier. If it is, the word is marked as QuantSub (line 13,14). Otherwise, we use NLTK [43] to judge the components of the word in the original sentence (line 15-25). We focus on five categories: nouns, pronouns, adjectives, adverbs and verbs. If both are nouns or pronouns, label it as ObjSub. If they are all adjectives or adverbs, mark it as NegaExp. If they are both verbs, tag it as Act Sub. Otherwise, it is assumed that we no longer need to consider it. When two sentences are unequal in length and W1 is completely contained within W2 (line 27), and if the missing part of W1 denotes a negation, e.g. not, do not, etc., this can be considered as the addition of a negative word, recorded as NegaExp (line 29). Otherwise, we assume this inconsistency is caused by WordDel (line 31).

In identifying sentence-level MR, as present in the previous section, back translation can be viewed as a free combination of word-level MRs. The most obvious feature of these use cases is that the sentences are close in length and the keywords in the sentences are basically the same. Modifiers in one sentence can often be found in another, or in the form of synonyms in another. Therefore, we screen the inconsistencies generated through back translation based on the above features. And in inference generation, the claim can usually be localised to a specific place in the context. In other words, there is a free compositional transformation relation between a claim sentence and the context. We consider this feature to identify pairs of sentences constructed by this relationship and generate negative sentences accordingly.

3.4 Triplet Completion

We generate the third sentence in triplets based on the two collected sentences. In total, this can be divided into three cases:

(1) Generating negative sentences by replacing quantifiers in base sentences.
(2) Generating positive sentences by rewriting base sentences.
(3) Generating negative sentences by destroying the inference relation between base and positive sentences.

In the case of the former, firstly, we need to recognize and extract the quantifiers in the sentence. This can be accomplished by employing a regular expression, which is a powerful text matching tool used to locate text fragments that conform to specific patterns. In this context, it's used to match quantifiers. Once we've successfully extracted the quantity from the sentence, the next step is to randomly adjust it. This means multiplying the original quantity by a random seed within the range of 0 to 2, excluding 1. Finally, we substitute the newly generated random number in place of the initial quantity within the sentence. This replacement effectively changes the sentence's meaning by altering the quantified value.

In the second case, we opt for the Text-Davinci-003 model. This model is well-suited for text generation tasks and is capable of comprehending and manipulating text effectively. We construct prompt word to achieve the desired transformation of the sentence. The prompt give the model hints and guidance on how to rewrite the sentence. They are designed to influence the model's output while keeping the core meaning intact. By employing this method, we can generate affirmative sentences that exhibit structural variations while retaining the original statement.

For the last case, we use the Davinci model for completion as well. More specifically, we construct the prompt words along the Chain-of-Thought [73] style. The purpose of the prompt word is to be a step-by-step guide for the model towards the target task.

Table 2: Models and Methods under test and corresponding Markers in the paper

Category	Sources/Types	Models/Methods & Marks			
Embedding Models	Open Source Projects or Vector Databases	Cohere/embed-english-v2.0 [13]	**Cohere-embed**	Paddle/ernie-3.0-medium-zh [70]	**Paddle-ernie**
		FastText-en [31]	**FastText**	sgugger/rwkv-430M-pile [23]	**Rwkv**
		Distilbert-base-uncased [57]	**DistilBERT**	all-MiniLM-L6-v2 [72]	**MiniLM**
		Paraphrase-albert-onnx [87]	**AlBERT-onnx**	unum-cloud/uform-vl-english [67]	**Uform**
		text-embedding-ada-002 [49]	**Ada**		
	SBERT's List or HuggingFace Model Hub	bert-base-uncased [17]	**BERT**	SpanBERT/spanbert-large-cased [30]	**SpanBERT**
		michiyasunaga/LinkBERT-base [47]	**LinkBERT**	google/electra-large-generator [12]	**Electra**
		xlnet-large-cased [81]	**XLNet**	sentence-transformers/gtr-t5-large [51]	**GTR-T5**
		roberta-base [42]	**RoBERTa**	sentence-transformers/sentence-t5-large [50]	**Sentence-T5**
		albert-base-v2 [34]	**AlBERT**	sentence-transformers/all-mpnet-base-v2 [48]	**MPNet**
	The latest LLMs	tiiuae/falcon-7b	**Falcon-7B**	tiiuae/falcon-7b-4bit	**Falcon-7B-Q**
		decapoda-research/llama-7b-hf	**LLaMA-7B**	decapoda-research/llama-7b-hf-4bit	**LLaMA-7B-Q**
		decapoda-research/llama-13b-hf	**LLaMA-13B**	decapoda-research/llama-13b-hf-4bit	**LLaMA-13B-Q**
		meta-llama/Llama-2-7b-hf	**LLaMA2-7B**	meta-llama/Llama-2-7b-hf-4bit	**LLaMA2-7B-Q**
		meta-llama/Llama-2-13b-hf	**LLaMA2-13B**	meta-llama/Llama-2-13b-hf-4bit	**LLaMA2-13B-Q**
Distance Metrics	Scipy	Cosine Distance	**CD**	Lance and Williams Distance	**LD**
		Euclidean Distance	**ED**	Pearson Correlation Distance	**PD**
		Manhalanobis Distance	**MD**	Manhattan Distance	**MhD**
		Bray Curtis Distance	**BD**		
Baselines	QG&QA-based	QuestEval	**QuestEval**		
	Cross-encoders	cross-encoder/quora-distilroberta-base	**DistilRoBERTa**		
		nli-deberta-v3-base	**DeBERTa**	nli-deberta-v3-base-reverse	**DeBERTa-R**
		roberta-large-mnli	**RoBERTa-mnli**	roberta-large-mnli-reverse	**RoBERTa-mnli-R**
	NLI models-based	summac-mnli-zs	**SummaC-zs**	summac-mnli-zs-reverse	**SummaC-zs-R**
		summac-mnli-conv	**SummaC-conv**	summac-mnli-conv-reverse	**SummaC-conv-R**

The process begins by directing the model to identify and focus on specific information pertaining to the supporting evidence that connects the claim to the surrounding context. Subsequently, we instruct the model to systematically remove this evidence-based information from the original context. The intention here is to disrupt the inferential relationship that previously existed. Until then, in this revised context, the inference of the claim would have been considered infeasible.

3.5 Simulation of Information Retrieval

In this section, we introduce how we measure the effectiveness of a vector matching method.

As shown in Figure 2, in this process, the first step involves the individual mapping of each sentence into a high-dimensional feature vector using a uniform embedding model. This mapping process facilitates the subsequent comparison of their similarity within a given feature space. This similarity is quantified using a particular distance metric, such as Euclidean Distance. We create both positive and negative vectors as the potential candidates. Using the base vector as our query vector, we emulate a scenario of retrieving information from a vector database. The goal is to find the item in the database that has the highest similarity to the query vector. We calculate the distances between the base vector and each of the two candidates separately. and the candidate vector with the smaller distance is then selected as the output of this matching operation. If the result point to the positive sentence, we consider the match correct. Conversely, we consider the matching erroneous.

4 EVALUATION

In this section, we evaluate whether MeTMaP can detect erroneous outputs. We simulate the process of vector matching in 203 (29 models and 7 distance metrics combined) vector matching methods for validation. We aim to answer the following research questions:

- **RQ1:** Are these test cases generated by our approach consistent with our requirements?
- **RQ2:** Is MeTMaP capable of identifying instances of false matching in vector databases
- **RQ3:** Can our approach find the erroneous outputs returned by information retrieval methods?
- **RQ4:** How different factors affect the performance of MeTMaP?

4.1 Experimental Settings

Dataset. We build a dataset containing 5,000 triplets for each MRs as data for our experiments. Initially, we gathered 5,000 sentences with quantifiers from the source datasets and generated corresponding positive and negative sentences. Next, we collected 5,000 implication pairs as positive sentences for Err Nli and formulated the negative sentences. For the remaining six MRs, we identified contradiction types in the source datasets to assemble base and negative sentences, generating positive sentences through rewriting. We get a corpus with eight sub-datasets, totalling 40,000 triplets. Furthermore, we modified the benchmark to simulate a non-metamorphic scenario. We maintain the base and positive sentences of the i th and $i + 1$ th (where i is odd) use cases but swap their negatives with the other's positive. In these test cases, both candidates differ structurally from the base sentence, yet only one preserves the same meaning. We conduct our experiments using the two corpora.
Models Under Test. To answer RQ2, we chose three vector databases integrated with Langchain for testing: Annoy [62], Chroma [11], ScaNN [24]. For simplifying RQ3's experimental process and encompassing diverse methods, we simulated vector matching. We

extracted embedding models used from the open source vector databases integrated with Langchain and projects in Github. For methods that provide a generic interface, we extracted the default model in the method. Additionally, we selected 10 models from SBERT's list and the Sentence-Transformers library of the HuggingFace Model Hub. We also evaluated the accuracy of popular and recent LLMs, including their 4-bit quantizations, like falcon [52] and llama [65, 66]. Details of all tested models and their marks in this study are provided in Table 2.

Distance Metrics. We focuses on seven metrics: Cosine Distance, Euclidean Distance, Manhalanobis Distance [4], Braycurtis Distance, Lance and Williams Distance, Pearson Correlation Distance and Manhattan Distance. Cosine and Euclidean Distances are particularly prevalent in vector databases like Milvus [69], FAISS [29], Hnswlib [46], PGvector [53], Chroma [11], DocArray [18], Pinecone [54]. All distance metrics are implemented by Scipy [61].

Baselines. Text matching methods require simultaneous input of two sentences to assess semantic differences effectively, which means that the model needs to consider the semantics of both sentences together rather than independently. We've chosen three types of text matching methods to calculate text similarity as our baselines. The first is QuestEval [60], an approach based on question generation and question answering (QG&QA). The second type involves cross-encoder models, like quora-distilroberta-base [57], nli-deberta-v3-base [26] and roberta-large-mnli [42]. The first model directly computes sentence similarity, while the latter two estimate probabilities of contradiction, neutral, and entailment. We take the probability of entailment as the similarity between texts. The third category includes extension methods based on NLI models, for which we selected SummaC [33] as a baseline.

Environment. All experiments are performed on a workstation with Ubuntu 20.04.3 LTS and 4 NVIDIA V100 GPUs (32GB memory).

4.2 Evaluation metrics

Distance. We measure sentence pair similarity using the distance between feature vectors. The distance from the base sentence to the positive sentence is noted as "positive distance", while its distance to the negative sentence is "negative distance". A smaller distance indicates higher similarity, with a distance of 0 signifying identical sentences. Ideally, the positive distance should be a number as small as possible, whereas the negative distance should be larger.

Scores. For those baselines, we interpret the scores generated by the methods as the similarity between two sentences. Scores associated with positive sentences are termed "positive scores", while those for negative sentences are "negative scores". Lower scores indicate greater similarity. We expect that positive scores will consistently be higher than negative scores.

Accuracy. In a test case, correct matching occurs if the method yields a smaller positive distance, or a higher positive score than the negative one. Incorrect matching happens in the opposite scenario. We evaluate a method's overall performance by tallying the number of correct matches across all test cases that should match.

Average distance/score. The representative distance/score for each dataset is determined by averaging the distances/scores between all sentence pairs in the set.

Table 3: The results of the case study

Vector Database	Acc without transformation	Acc under MeTMaP
Annoy	99.72	37.72
Chroma	70.28	23.80
ScaNN	99.72	37.72

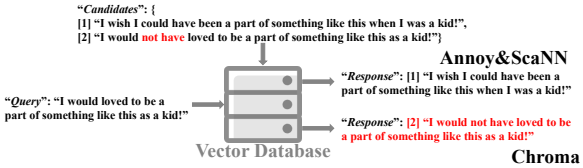


Figure 3: An example of a real problem in a vector database.

Note that all distances are calculated after normalizing the embedding vectors. All distances and scores in the experimental results are Max-Min normalized.

4.3 RQ1: Are these test cases generated by our approach consistent with our requirements?

MeTMaP aim to construct complete triplets as test examples. Therefore, in this section, we evaluate whether the triplets satisfy the requirement of Equation 1. We randomly sampled 100 cases from each dataset, creating 800 triplets with shuffled sentence orders. We labelled their semantic and structural similarities separately. We recruit three annotators with a Bachelor's degree or above and proficiency in both Chinese and English. The final labels are determined by majority vote. At first, annotators are asked to identify which two sentences had similar meanings. Finally, in 93.63% of cases, the base and positive sentences are considered to be more semantically similar, aligning the semantic consistency requirement.

After further shuffling the order within and between triplets, we posed a second question to them: Which two sentences do you think have a more similar sentence structure? We expected the base and negative sentences to be indexed as most structurally similar. With the exception of Err Nli, where negatives are derived from positives. The results align with our expectations in 92.75% of the test cases, indicating that, in terms of sentence structure, the base sentences are typically more similar to the negative sentences than to the positive ones.

A test case is deemed compliant if the responses to both questions align with our expectations. Our analysis shows that 86.5% of the cases meet these criteria, suggesting that the majority of MeTMaP triplets fulfill our construction objectives. The non-compliant fraction likely exhibits weaker semantic or structural properties, but not to the extent of complete incompatibility with our requirements. We excluded these cases from further experiments.

RQ1 Answer: The dataset constructed by MeTMaP satisfies both structural and semantic requirements.

4.4 RQ2: Is MeTMaP capable of identifying instances of false matching in vector databases

In this section, we conduct a case study to identify false matching problems in open source vector databases. The results of the study

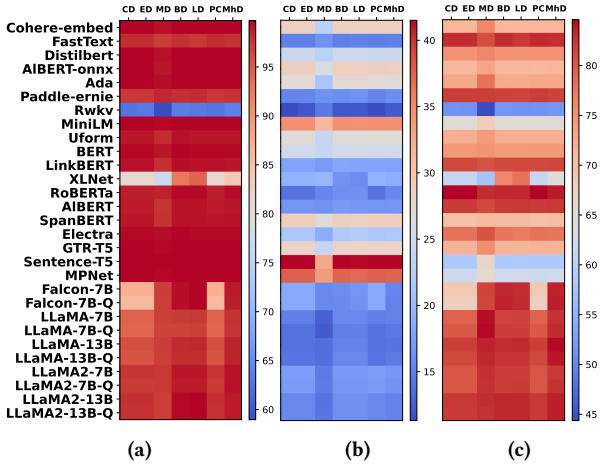


Figure 4: The effect of metamorphosis on accuracy. Figure 4a shows the accuracy without transformation. Figure 4b gives the accuracy under MeTMaP. Figure 4c is a statistic of the decrease in accuracy of the methods before and after the transformation.

are shown in Table 3. We observe that the query results of the vector databases demonstrate a high accuracy when no transformation are applied to the retrieved candidates. However, when candidates conform to our triplet specification, there is a noticeable drop in the accuracy of the database retrieval. An example of a real problem found in Chroma, is illustrated in Figure 3. In this case, the negative sentence, formed by adding "not have" to the base sentence to convey opposite semantics, is incorrectly matched. The response results from databases show that Annoy and ScaNN choose *candidate-1* with the same semantics, whereas Chroma considers *candidate-2* to be closer to the query, indicating a false matching problem.

Additionally, as Annoy and ScaNN utilize identical default embedding model and distance metric, they yield the same results. This suggest that components external to the matching method in these databases have minimal impact on retrieval outcomes.

RQ2 Answer: MeTMaP is effective in detecting a numerous false matching problems in real vector databases. The accuracy of these databases' retrieval largely depends on the vector matching methods employed, rather than on external integration components.

4.5 RQ3: Can our approach find the erroneous outputs returned by the information retrieval methods?

MeTMaP is designed to construct triplet test cases to evaluate the accuracy of a specific information retrieval methods. We combine the embedding models and the distance metrics in pairs as various vector matching methods. Recognizing that developers might experiment with a range of combinations, even those not typically suited for practical vector matching, in this section, we evaluate 203 different vector matching methods. We quantify the matching performance of these methods on the entire dataset.

Figure 4a demonstrates that without additional metamorphosis, all methods exhibit high accuracy, with the lowest being 58.97%.

Table 4: Accuracy of the Baselines on the total dataset

Methods	Default	Reverse
QuestEval	0.56	-
DistilRoBERTa	0.78	-
DeBERTa	0.89	0.89
RoBERTa-mnli	0.80	0.82
SummaC-zs	0.79	0.82
SummaC-conv	0.65	0.74

Remarkably, 177 methods achieve over 95% accuracy, and 185 surpass 90%. However, their performance drops significantly with the inclusion of disturbing sentences in test cases. As indicated in Figure 4b and Figure 4c, the average accuracy decreases by more than 78.07%, with the highest accuracy among all combinations being only 41.51%. Notably, 120 methods fall below 20% accuracy. These findings suggest that MeTMaP successfully uncovers numerous erroneous matching results produced by these methods.

Comparing with vector matching methods, these baselines demonstrate superior performance. Table 4 shows that all six methods, along with their variants featuring reversed input order, outperform vector matching methods. Notably, eight of these achieve over 70% accuracy even with interference, and five exceed 80%. DeBERTa and its variant display an accuracy rate of 89%, the best among all methods. These results further underscore MeTMaP's effectiveness in detecting erroneous outputs in vector matching methods.

RQ3 Answer: Under the test of MeTMaP, all 203 vector matching methods have an accuracy of no more than 42%, and 120 of them falling below 20%. In contrast, the baselines performed better, with eight methods reaching 70% or higher, and the best performer achieving 89%.

4.6 RQ4: How different factors affect the performance of MeTMaP?

As shown in Figure 5, when MeTMaP employs different MRs for metamorphosis, the vector matching methods produce significant variation in the number of erroneous outputs. Specifically, the distribution of average negative distances shows remarkable similarity across the seven distance metrics. Notably, the lowest values frequently appear in Word Swap, Obj Sub, Quant Sub, and Act Sub categories. For instance, the percentage of minima occurring in these four categories is 72.41%, 41.38%, 55.17%, 34.48% in CD and 55.17%, 27.59%, 31.03% and 27.59% in MD, respectively. Furthermore, in word-level MRs, positive mean distances are typically exceed negative mean distances.

Additionally, Figure 5 also indicates a strong correlation between the lowest matching accuracy of each model and specific categories. Specifically, lower average negative distances often correspond to lower accuracy. We consider that this trend relates to the degree of structural change induced by metamorphosis. In Word Swap, where only two words are swapped without altering sentence structure or introducing new words, the metamorphosis is minimal. In contrast, MRs based on word substitution, while not altering structure, introduce new words, resulting in more significant metamorphosis compared to Word Swap. The most substantial structural changes at the word level occur in Nega Exp and Word Del, as these involve adding negatives or deleting parts of sentences. At the sentence

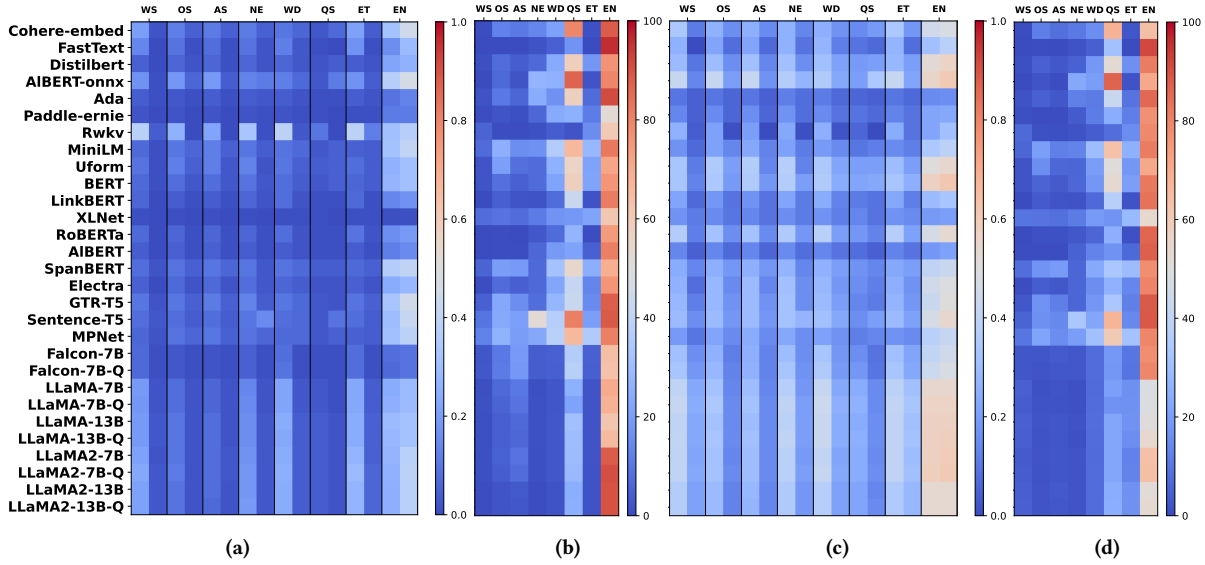


Figure 5: Average positive and negative distances and accuracies on all sub-datasets.Figure 5a shows the average positive and negative distances for the CD-based methods, where the left side of each column represents the positive one, and the right is the negative one. Figure 5b is the accuracy of all CD-based methods on sub-datasets. Figure 5c shows the average distances for the MD-based methods. Figure 5d is the accuracy of all MD-based methods on sub-datasets.

level, Back Translation usually involves a combination of multiple word level MRs, resulting in more extensive structural changes than those at the word level. In Inference Generation, base and positive sentences differ significantly in structure as they are connected by inference. These difference between base and negative sentence is further amplified by removing evidence supporting the base sentence from the positive sentence. We believe that is the highest negative average distances and peak matching accuracies consistently occur in this category across all distance metrics.

Overall, we consider that vector matching methods are more sensitive to structural variations in sentences than to semantic differences. MeTMaP detects a higher number of erroneous outputs when evaluating pairs with less structural metamorphosis.

As discussed in section 3, Word Del and Err Nli exhibit *One-Way Inconsistency* between base sentences and their respective negative and positive sentences. These inconsistencies are detectable only in a specific direction. Vector matching methods, however, tend to overlook these nuances due to their disregard for input order. We expect these methods, which have requirements on the order of inputs, to detect these problems. For methods sensitive to input sequence, we frame their task as determining the likelihood of *sentence1* entailing *sentence2*, assuming *sentence1* is the first input.

As we see in Figure 6, NLI model-based methods and cross-encoder models typically yield higher negative scores for Word Del and lower positive scores for Err Nli, with the exception of DistilRoBERTa. This is due to the fact that DistilRoBERTa differs as it evaluates the similarity of two sentences regardless of their input order. Variants using a consistent-with-expectations approach produce contrasting results, indicating their capability to detect one-way inconsistency. This also highlights that NLI-based text matching methods are more adept at uncovering semantic differences. Although inconsistencies can be detected, it is the positive

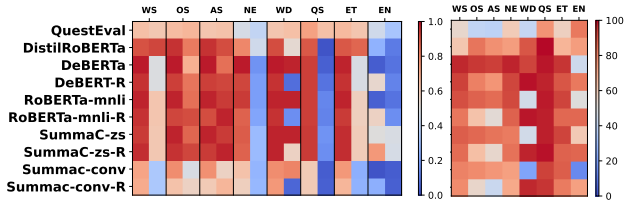


Figure 6: Average positive and negative scores and accuracies of Baselines on all sub-datasets. Figure 6a shows the average positive and negative scores, where the left side of each column represents the positive one, and the right is the negative one. Figure 6b is the accuracy of baselines.

sentence that should be closer to the base one in Word Del cases, as it encompass a comprehensive semantics of the base sentence.

RQ4 Answer: MeTMaP identifies more erroneous outputs when testing with cases constructed via minor metamorphosis. Additionally, by specifying the input order of sentences, MeTMaP uncovers new erroneous outputs by revealing one-way inconsistencies in specific scenarios.

5 DISCUSSIONS

Limitations on proposed metamorphic relations. A limitation may arise from the use of MRs derived from prominent datasets, which may not fully encompass all word and sentence-level metamorphosis. The metamorphosis we chose intentionally minimally alter sentence structure to gauge their impact on information retrieval accuracy. This focus on subtle shifts aims to reveal how

minor variations can affect retrieval robustness and efficiency, despite limiting the range of metamorphosis types considered.

Sensitivity to MRs. Our study might also be limited by the varying sensitivities of models to different MRs. Some models may be better at detecting certain metamorphosis than others. Nonetheless, our study's primary focus was on the relations themselves. The differential sensitivity to these relations, while an important aspect, is considered orthogonal to our primary scope.

Potential for Triplet Generation. Currently, only the triplet corresponding to Quant Sub is generated from a base sentence. However, there is significant potential to extend this functionality to include triplet generation for other categories. Our goal is to develop a system capable of automatically generating multiple triplets for various MRs from any given sentence, thus greatly enhancing the robustness and versatility of our approach.

Text-matching versus Vector-matching. Our research highlights a key limitation in the trade-off between text-matching and vector matching methods. Text-matching offers higher accuracy but is time-intensive, as it calculates similarity between the query and every sentence in the database for each retrieval. Conversely, Vector matching is quicker and easier to implement but sometimes less accurate. We propose a future solution combines both methods: using vector matching first to quickly identify a subset of potential matches, followed by text-matching for precise final match identification. This hybrid approach aims to balance speed and precision, leveraging the advantages of both methods.

False Matching Mitigation. Following discussions with vector database developers about false match issues, we're exploring a hybrid approach combining vector and text matching. This method first groups candidates via vector matching, then applies text matching within the most similar group for accuracy. However, it's important to note that this solution requires the databases to support both vector and text matching methods for retrieval.

6 THREATS TO VALIDITY

Internal Threat. The main internal concern regarding our study's validity comes from the reliability of our dataset. Despite rigorous efforts by three volunteers to accurately label each sentence in the dataset, bias and inaccuracies may still occur. Such issues, even if minor, could affect our analysis and findings. Recognizing this, future work should seek more diverse labeling methods or automated checks to minimize these issues.

External Threat. The major external threat to our study's validity is the loss of precision from model quantization. To manage large models on limited video memory, we used a 4-bit quantization technique. However, this approach inherently leads to a loss of precision, which can adversely affect the method's accuracy. This process might introduce errors, affecting the generalizability and accuracy of our results on models not subjected to this quantization.

7 RELATED WORK

7.1 Testing NLP Applications

NLP applications are widely used in tasks such as information retrieval [35, 68, 82], sentiment analysis [19, 56, 76], virtual assistants [25, 32, 85], and text/code summarization [21, 44, 80], prompting research on their robustness and reliability. Unlike existing studies, our work focuses on the under-explored areas of LLM-augmented generation regarding these aspects.

Robustness. Ribeiro et al. [55] created a behavioral testing method for NLP, targeting sentiment analysis, duplicate question answering, and comprehension. Li et al. [37] used deep learning to create test cases for NLP applications. Sun et al. [63] presented a word replacement method to fix machine translation mistakes without retraining. Xiao et al. [78] introduced an automated testing method that uses LEvy flight-based Adaptive Particle Swarm Optimization and text analysis for adversarial scenario creation. Wang et al. [71] examined the robustness of widely-used content moderation tools.

Reliability. Tan et al. [64] propose a framework for realiability testing in NLP systems, using adversarial attacks and interdisciplinary collaboration to uphold industry standards. Chen et al. [7] introduce a label-independent testing method for QA software through Metamorphic Relations applied to recursive questions, uncovering prevalent answering flaws. Alshemali et al. [3] reviews the vulnerability of deep neural networks in NLP to adversarial examples, categorizes methods for generating adversarial texts, and explores defensive strategies and challenges.

7.2 Testing LLM-integrated Applications

[27] explored the intersection of LLMs and software engineering. Testing LLM-integrated applications is fraught with challenges and complexities,exacerbated by the widespread use of OpenAI's GPT API. Tools like Langchain [6], ChatDB [28], LlamaIndex [39], and Flowise [20] simplify the development and testing of LLM-integrated applications by converting language into more models.

Prior research has primarily focused on identifying and mitigating potential attack vectors on these applications, including "jailbreaking" [2, 5, 15, 16, 38, 41, 79] to bypass safeguards, hijacking prompt intent, leaking prompts, vulnerability to backdoor attacks with tainted datasets, and "prompt injections" [22, 40] for executing unauthorized commands. Additionally, the issue of copyright in LLM training data has been examined [36]. Our work adds to this by exploring the dynamics between LLMs and their integrated memory, particularly methods for precise information retrieval.

8 CONCLUSION

This paper introduces a groundbreaking metamorphic testing framework tailored for LLM augmented generation, pioneering the evaluation of vector matching techniques in this context. The comprehensive assessment of 203 vector matching methods reveals a notable bias towards structural similarity, underscoring a critical area for improvement in semantic accuracy. These findings not only highlight the existing challenges in LLM augmentation but also provide a clear direction for future enhancements. As we continue to refine these techniques, our work lays a foundational path towards developing more accurate, reliable, and semantically sophisticated LLM augmented systems.

ACKNOWLEDGEMENTS

This research has been supported by the National Natural Science Foundation of China (grant No.62302176).

REFERENCES

[1] 2023. MeTMaP. https://anonymous.4open.science/r/MeTMaP-879B. (2023).
[2] 0xk1h0. 2023. ChatGPT_DAN. https://github.com/0xk1h0/ChatGPT_DAN. (2023).
[3] Basemah Alshemali and Jugal Kalita. 2020. Improving the reliability of deep neural networks in NLP: A review. *Knowledge-Based Systems* 191 (2020).
[4] Mahalanobis Prasanta Chandra et al. 1936. On the generalised distance in statistics. In *Proceedings of the National Institute of Sciences of India*, Vol. 2. 49–55.
[5] Zhiyuan Chang, Mingyang Li, Yi Liu, Junjie Wang, Qing Wang, and Yang Liu. 2024. Play Guessing Game with LLM: Indirect Jailbreak Attack with Implicit Clues. *arXiv preprint arXiv:2402.09091* (2024).
[6] Harrison Chase. 2022. LangChain. https://python.langchain.com/docs/get_started/introduction. (2022).
[7] Songqiang Chen, Shuo Jin, and Xiaoyuan Xie. 2021. Testing Your Question Answering Software via Asking Recursively. In *ASE*. 104–116. https://doi.org/10.1109/ASE51524.2021.9678670
[8] Tsong Yueh Chen, S. C. Cheung, and Siu-Ming Yiu. 2020. Metamorphic Testing: A New Approach for Generating Next Test Cases. *ArXiv* abs/2002.12543 (2020). https://api.semanticscholar.org/CorpusID:15467386
[9] Tsong Yueh Chen, Fei-Ching Kuo, Huai Liu, Pak-Lok Poon, Dave Towey, T. H. Tse, and Zhi Quan Zhou. 2018. Metamorphic Testing: A Review of Challenges and Opportunities. *ACM Comput. Surv.* 51, 1 (jan 2018), 27.
[10] Cheng-Han Chiang, Yung-Sung Chuang, James Glass, and Hung-yi Lee. 2023. Revealing the Blind Spot of Sentence Encoder Evaluation by HEROS. *arXiv preprint arXiv:2306.05083* (2023).
[11] chroma core. 2023. Chroma. https://github.com/chroma-core/chroma. (2023).
[12] Kevin Clark, Minh-Thang Luong, Quoc V Le, and Christopher D Manning. 2020. Electra: Pre-training text encoders as discriminators rather than generators. *arXiv preprint arXiv:2003.10555* (2020).
[13] cohere. 2023. Cohere. https://dashboard.cohere.com/. (2023).
[14] Marie-Catherine De Marneffe, Anna N Rafferty, and Christopher D Manning. 2008. Finding contradictions in text. In *Proceedings of acl-08: Hlt*. 1039–1047.
[15] Gelei Deng, Yi Liu, Yuekang Li, Kailong Wang, Ying Zhang, Zefeng Li, Haoyu Wang, Tianwei Zhang, and Yang Liu. 2023. Jailbreaker: Automated Jailbreak Across Multiple Large Language Model Chatbots. (2023). arXiv:cs.CR/2307.08715
[16] Gelei Deng, Yi Liu, Kailong Wang, Yuekang Li, Tianwei Zhang, and Yang Liu. 2024. Pandora: Jailbreak GPTs by Retrieval Augmented Generation Poisoning. *arXiv preprint arXiv:2402.08416* (2024).
[17] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *CoRR* abs/1810.04805 (2018). arXiv:1810.04805 http://arxiv.org/abs/1810.04805
[18] docarray. 2023. DocArray. https://github.com/docarray/docarray. (2023).
[19] Salvatore Claudio Fanni, Maria Febi, Gayane Aghakhanyan, and Emanuele Neri. 2023. Natural language processing. In *Introduction to Artificial Intelligence*. 87–99.
[20] FlowiseAI. 2023. Flowise. https://github.com/FlowiseAI/Flowise. (2023).
[21] Mingyang Geng, Shangwen Wang, Dezun Dong, Haotian Wang, Shaomeng Cao, Kechi Zhang, and Zhi Jin. 2023. Interpretation-based Code Summarization. In *ICPC*.
[22] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. (2023). arXiv:cs.CR/2302.12173
[23] Sylvain Gugger. 2023. RWKV. https://huggingface.co/sgugger/rwkv-430M-pile. (2023).
[24] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. Accelerating Large-Scale Inference with Anisotropic Vector Quantization. In *International Conference on Machine Learning*. https://arxiv.org/abs/1908.10396
[25] Walid Hariri. 2023. Unlocking the Potential of ChatGPT: A Comprehensive Exploration of its Applications, Advantages, Limitations, and Future Directions in Natural Language Processing. *arXiv preprint* (2023).
[26] Pengcheng He, Jianfeng Gao, and Weizhu Chen. 2021. DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing. (2021). arXiv:cs.CL/2111.09543
[27] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2023. Large Language Models for Software Engineering: A Systematic Literature Review. (2023). arXiv:cs.SE/2308.10620
[28] Chenxu Hu, Jie Fu, Chenzhuang Du, Simian Luo, Junbo Zhao, and Hang Zhao. 2023. ChatDB: Augmenting LLMs with Databases as Their Symbolic Memory. (2023). arXiv:cs.AI/2306.03901
[29] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547.
[30] Mandar Joshi, Danqi Chen, Yinhan Liu, Daniel S Weld, Luke Zettlemoyer, and Omer Levy. 2020. Spanbert: Improving pre-training by representing and predicting spans. *Transactions of the association for computational linguistics* 8 (2020), 64–77.
[31] Armand Joulin, Edouard Grave, Piotr Bojanowski, Matthijs Douze, Hérve Jégou, and Tomas Mikolov. 2016. FastText.zip: Compressing text classification models. *arXiv preprint arXiv:1612.03651* (2016).
[32] Sana Zehra Kamoonpuri and Anita Sengar. 2023. Hi, May AI help you? An analysis of the barriers impeding the implementation and use of artificial intelligence-enabled virtual assistants in retail. *Journal of Retailing and Consumer Services* 72 (2023).
[33] Philippe Laban, Tobias Schnabel, Paul N Bennett, and Marti A Hearst. 2022. SummaC: Re-visiting NLI-based models for inconsistency detection in summarization. *Transactions of the Association for Computational Linguistics* 10 (2022), 163–177.
[34] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. 2019. ALBERT: A Lite BERT for Self-supervised Learning of Language Representations. *CoRR* abs/1909.11942 (2019). arXiv:1909.11942 http://arxiv.org/abs/1909.11942
[35] David D Lewis and Karen Spärck Jones. 1996. Natural language processing for information retrieval. *Communications of the ACM* 39, 1 (1996), 92–101.
[36] Haodong Li, Gelei Deng, Yi Liu, Kailong Wang, Yuekang Li, Tianwei Zhang, Yang Liu, Guoai Xu, Guosheng Xu, and Haoyu Wang. 2024. Digger: Detecting Copyright Content Mis-usage in Large Language Model Training. (2024). arXiv:cs.CR/2401.00676
[37] Jinfeng Li, Shouling Ji, Tianyu Du, Bo Li, and Ting Wang. 2019. TextBugger: Generating Adversarial Text Against Real-world Applications. In *NDSS*.
[38] Jie Li, Yi Liu, Chongyang Liu, Ling Shi, Xiaoning Ren, Yaowen Zheng, Yang Liu, and Yinxing Xue. 2024. A Cross-Language Investigation into Jailbreak Attacks in Large Language Models. *arXiv preprint arXiv:2401.16765* (2024).
[39] Jerry Liu. 2022. LlamaIndex. (11 2022). https://doi.org/10.5281/zenodo.1234
[40] Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Tianwei Zhang, Yepang Liu, Haoyu Wang, Yan Zheng, and Yang Liu. Prompt Injection attack against LLM-integrated Applications, June 2023. *arXiv preprint arXiv:2306.05499* (????).
[41] Y Liu, G Deng, Z Xu, Y Li, Y Zheng, Y Zhang, L Zhao, T Zhang, and Y Liu. Jailbreaking chatgpt via prompt engineering: An empirical study (2023). *Preprint at https://arxiv. org/abs/2305.13860* (????).
[42] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692* (2019).
[43] Edward Loper and Steven Bird. 2002. Nltk: The natural language toolkit. *arXiv preprint cs/0205028* (2002).
[44] Zheheng Luo, Qianqian Xie, and Sophia Ananiadou. 2023. Chatgpt as a factual inconsistency evaluator for abstractive text summarization. *arXiv preprint* (2023).
[45] Pingchuan Ma, Shuai Wang, and Jin Liu. 2020. Metamorphic Testing and Certified Mitigation of Fairness Violations in NLP Models. In *International Joint Conference on Artificial Intelligence*. https://api.semanticscholar.org/CorpusID:220483049
[46] Yu A Malkov and Dmitry A Yashunin. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence* 42, 4 (2018), 824–836.
[47] Michihiro Yasunaga and Jure Leskovec and Percy Liang. 2022. LinkBERT: Pretraining Language Models with Document Links. In *ACL*.
[48] microsoft. 2023. MPNet. https://huggingface.co/microsoft/mpnet-base. (2023).
[49] Arvind Neelakantan, Tao Xu, Raul Puri, Alec Radford, Jesse Michael Han, Jerry Tworek, Qiming Yuan, Nikolas Tezak, Jong Wook Kim, Chris Hallacy, et al. 2022. Text and code embeddings by contrastive pre-training. *arXiv preprint arXiv:2201.10005* (2022).
[50] Jianmo Ni, Gustavo Hernández Ábrego, Noah Constant, Ji Ma, Keith B Hall, Daniel Cer, and Yinfei Yang. 2021. Sentence-t5: Scalable sentence encoders from pre-trained text-to-text models. *arXiv preprint arXiv:2108.08877* (2021).
[51] Jianmo Ni, Chen Qu, Jing Lu, Zhuyun Dai, Gustavo Hernández Ábrego, Ji Ma, Vincent Y Zhao, Yi Luan, Keith B Hall, Ming-Wei Chang, et al. 2021. Large dual encoders are generalizable retrievers. *arXiv preprint arXiv:2112.07899* (2021).
[52] Guilherme Penedo, Quentin Malartic, Daniel Hesslow, Ruxandra Cojocaru, Alessandro Cappelli, Hamza Alobeidli, Baptiste Pannier, Ebtesam Almazrouei, and Julien Launay. 2023. The RefinedWeb dataset for Falcon LLM: outperforming curated corpora with web data, and web data only. *arXiv preprint arXiv:2306.01116* (2023). arXiv:2306.01116 https://arxiv.org/abs/2306.01116
[53] pgvector. 2023. PGVector. https://github.com/pgvector/pgvector. (2023).
[54] pinecone. 2023. Pinecone. https://www.pinecone.io/. (2023).
[55] Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. 2020. Beyond Accuracy: Behavioral Testing of NLP Models with CheckList. In *ACL*. 4902–4912.
[56] Abdul Rahaman Wahab Sait and Mohamad Khairi Ishak. 2023. Deep learning with natural language processing enabled sentimental analysis on sarcasm classification. *Comput. Syst. Sci. Eng* 44, 3 (2023), 2553–2567.
[57] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *ArXiv* abs/1910.01108 (2019).
[58] Sebastião Santos, Beatriz Nogueira Carvalho da Silveira, Stevão Alves de Andrade, Márcio Eduardo Delamaro, and Simone do Rocio Senger de Souza. 2020. An

Experimental Study on Applying Metamorphic Testing in Machine Learning Applications. *Proceedings of the 5th Brazilian Symposium on Systematic and Automated Software Testing* (2020). https://api.semanticscholar.org/CorpusID:225040791

[59] Tal Schuster, Adam Fisch, and Regina Barzilay. 2021. Get your vitamin C! robust fact verification with contrastive evidence. *arXiv preprint arXiv:2103.08541* (2021).

[60] Thomas Scialom, Paul-Alexis Dray, Patrick Gallinari, Sylvain Lamprier, Benjamin Piwowarski, Jacopo Staiano, and Alex Wang. 2021. Questeval: Summarization asks for fact-based evaluation. *arXiv preprint arXiv:2103.12693* (2021).

[61] scipy. 2023. Fundamental algorithms for scientific computing in Python. https://scipy.org/. (2023).

[62] Spotify. 2023. Annoy. https://github.com/spotify/annoy?tab=readme-ov-file. (2023).

[63] Zeyu Sun, J Zhang, Yingfei Xiong, Mark Harman, Mike Papadakis, and Lu Zhang. 2022. Improving Machine Translation Systems via Isotopic Replacement. In *ICSE*. 1181–1192.

[64] Samson Tan, Shafiq Joty, Kathy Baxter, Araz Taeihagh, Gregory A. Bennett, and Min-Yen Kan. 2021. Reliability Testing for Natural Language Processing Systems. (2021). arXiv:cs.LG/2105.02590

[65] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).

[66] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).

[67] unum cloud. 2023. Uform. https://huggingface.co/unum-cloud/uform-vl-english. (2023).

[68] Ellen M Voorhees. 1999. Natural language processing and information retrieval. In *International summer school on information extraction*. 32–48.

[69] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, et al. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the 2021 International Conference on Management of Data*. 2614–2627.

[70] Shuohuan Wang, Yu Sun, Yang Xiang, Zhihua Wu, Siyu Ding, Weibao Gong, Shikun Feng, Junyuan Shang, Yanbin Zhao, Chao Pang, et al. 2021. Ernie 3.0 titan: Exploring larger-scale knowledge enhanced pre-training for language understanding and generation. *arXiv preprint arXiv:2112.12731* (2021).

[71] Wenxuan Wang, Jen-tse Huang, Weibin Wu, Jianping Zhang, Yizhan Huang, Shuqing Li, Pinjia He, and Michael R. Lyu. 2023. MTTM: Metamorphic Testing for Textual Content Moderation Software. In *ICSE*. 2387–2399.

[72] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. 2020. Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. *Advances in Neural Information Processing Systems* 33 (2020), 5776–5788.

[73] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems* 35 (2022), 24824–24837.

[74] Orion Weller, Dawn Lawrie, and Benjamin Van Durme. 2023. NevIR: Negation in Neural Information Retrieval. *arXiv preprint arXiv:2305.07614* (2023).

[75] Aaron Steven White, Pushpendre Rastogi, Kevin Duh, and Benjamin Van Durme. 2017. Inference is everything: Recasting semantic resources into a unified evaluation framework. In *Proceedings of the Eighth International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. 996–1005.

[76] P William, Anurag Shrivastava, Premanand S Chauhan, Mudasir Raja, Sudhir Baijnath Ojha, and Keshav Kumar. 2023. Natural Language processing implementation for sentiment analysis on tweets. In *MRCN*. 317–327.

[77] Dongwei Xiao, Zhibo Liu, Yuanyuan Yuan, Qi Pang, and Shuai Wang. 2022. Metamorphic Testing of Deep Learning Compilers. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 6 (2022), 1 – 28. https://api.semanticscholar.org/CorpusID:247159402

[78] Mingxuan Xiao, Yan Xiao, Hai Dong, Shunhui Ji, and Pengcheng Zhang. 2023. LEAP: Efficient and Automated Test Method for NLP Software. *arXiv preprint arXiv:2308.11284* (2023).

[79] Zihao Xu, Yi Liu, Gelei Deng, Yuekang Li, and Stjepan Picek. 2024. LLM Jailbreak Attack versus Defense Techniques–A Comprehensive Study. *arXiv preprint arXiv:2402.13457* (2024).

[80] Xianjun Yang, Yan Li, Xinlu Zhang, Haifeng Chen, and Wei Cheng. 2023. Exploring the limits of chatgpt for query or aspect-based text summarization. *arXiv preprint* (2023).

[81] Zhilin Yang, Zihang Dai, Yiming Yang, Jaime G. Carbonell, Ruslan Salakhutdinov, and Quoc V. Le. 2019. XLNet: Generalized Autoregressive Pretraining for Language Understanding. *CoRR* abs/1906.08237 (2019). arXiv:1906.08237 http://arxiv.org/abs/1906.08237

[82] Arister NJ Yew, Marijn Schraagen, Willem M Otte, and Eric van Diessen. 2023. Transforming epilepsy research: A systematic review on natural language processing applications. *Epilepsia* 64, 2 (2023), 292–305.

[83] Yuanyuan Yuan, Qi Pang, and Shuai Wang. 2022. Unveiling Hidden DNN Defects with Decision-Based Metamorphic Testing. *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering* (2022). https://api.semanticscholar.org/CorpusID:252816122

[84] Yuan Zhang, Jason Baldridge, and Luheng He. 2019. PAWS: Paraphrase adversaries from word scrambling. *arXiv preprint arXiv:1904.01130* (2019).

[85] Shuo Zhou, Joshva Silvasstar, Christopher Clark, Adam J Salyers, Catia Chavez, and Sheana S Bull. 2023. An artificially intelligent, natural language processing chatbot designed to promote COVID-19 vaccination: A proof-of-concept pilot study. *Digital Health* 9 (2023).

[86] zilliztech. 2023. GPTCache. https://github.com/zilliztech/GPTCache. (2023).

[87] zilliztech. 2023. Paraphrase-albert-onnx. https://huggingface.co/GPTCache/paraphrase-albert-onnx. (2023).